UNIT - 7
PERFORMANCE EVALUATION in R

Asst. Prof. Prashant Kothari
Assistant Professor
Computer Science and Engineering

Contents

1. Loss Function and Error: Mean Squared Error
2. Root Mean Squared Error
3. Model Selection and Evaluation criteria: Accuracy, Precision, F1 score, Recall Score
4. Binary Predictive Classification
5. Sensitivity
6. Specificity

Performance Evaluation in R

Performance Evaluation in Machine Learning means **measuring how well a trained model performs on data.** It helps us determine whether the model is making **accurate predictions** or **large errors**.

Two important concepts are:

1. **Loss Function**
2. **Error**

These concepts are especially important in **Regression** and **Classification**.

What is Performance Evaluation?

Suppose we build a model to predict house prices.

Actual values: Actual Price = ₹50 lakh

Model prediction: Predicted Price = ₹45 lakh

The difference is:

Error = Actual - Predicted

Error = 50 - 45 lakh

Error = **₹5 lakh**

Performance evaluation helps us answer: **How close are the model's predictions to the actual values?**

What is Performance Evaluation?

Basic Process

Training Data

↓

Machine Learning Model

↓

Predictions

↓

Compare with Actual Values

↓

Calculate Error/Loss

↓

Evaluate Model Performance

What is Error?

Error is the difference between the **actual value** and the **predicted value**.

The basic formula is:

$$\text{Error} = \text{Actual} - \text{Predicted}$$

Example –

Suppose

Actual = 100

Predicted = 90 Then:

Error = 100 − 90

Error = 10

So the prediction **error** is **10**.

Positive and Negative Error

Error can be **positive** or **negative**.

Example 1

Actual = 100

Predicted = 90

Error = 100 − 90

Error = 10

Positive **error** = **10**

Positive and Negative Error

Error can be **positive** or **negative**.

Example 2

Actual = 100

Predicted = 110

Error = 100 − 110

Error = -10

Negative **error** = **-10**

A negative error means the model **overestimated** the actual value.

Loss Function

A Loss Function is a **mathematical function** that measures **how much a model's prediction differs from the actual value**.

In simple words- Loss function tells us how wrong the model's prediction is.

Generally:

Small Loss → Better Prediction

Large Loss → Poor Prediction

Error vs Loss Function

Error	Loss Function
Difference between actual and predicted value	Function used to measure prediction loss
Can be positive or negative	Usually designed to produce a non-negative loss
Describes an individual prediction error	Can summarize the error according to a chosen mathematical rule

Error vs Loss Function

For example:

Actual = 100

Predicted = 90

Error = 100 - 90

Error = 10

Squared Loss = $(100 - 90)^2$

Squared Loss = 100

Common Loss Functions for Regression

For regression problems, some commonly used loss/error measures are:

1. **Mean Squared Error (MSE)**
2. **Root Mean Squared Error (RMSE)**
3. **Mean Absolute Error (MAE)**
4. **Mean Absolute Percentage Error (MAPE)**

Mean Squared Error (MSE)

MSE Mean Squared Error (MSE) is a commonly used **loss function** and **evaluation metric** for **regression models**. It measures how far the predicted values are from the actual values.

In simple words:

MSE tells us the **average squared difference between actual values** and **predicted values**.

A **smaller MSE** generally means the **predictions are closer to the actual values**.

Mean Squared Error (MSE)

MSE Formula

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- n = Number of observations
- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- $y_i - \hat{y}_i$ = Error

Root Mean Squared Error (RMSE)

Mean Squared Error, Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

RMSE is the square root of the average squared difference between actual and predicted values.

Model Selection and Evaluation criteria

In Machine Learning, after training a classification model, we need to determine **how well the model performs on unseen data**. This process is called **model evaluation**.

For classification problems, some of the most commonly used evaluation metrics are:

1. **Accuracy**
2. **Precision**
3. **Recall**
4. **F1-Score**

These metrics are calculated using the **Confusion Matrix.**

Confusion Matrix

Before understanding Accuracy, Precision, Recall, and F1-Score, we need to understand four basic terms.

Actual / Predicted	Positive	Negative
Positive	True Positive (TP)	False Negative (FN)
Negative	False Positive (FP)	True Negative (TN)

Confusion Matrix

True Positive (TP)

The model predicts **Positive**, and the actual class is also **Positive**.

Example: A fraudulent transaction is correctly identified as fraud.

True Negative (TN)

The model predicts **Negative**, and the actual class is also **Negative**.

Example: A genuine transaction is correctly identified as genuine.

Confusion Matrix

False Positive (FP)

The model predicts **Positive**, but the actual class is **Negative**.

Example: A genuine transaction is incorrectly classified as fraud.

False Negative (FN)

The model predicts **Negative**, but the actual class is **Positive**.

Example: A fraudulent transaction is incorrectly classified as genuine.

Accuracy

Accuracy measures the overall percentage of predictions that are correctly classified by the model.

In other words - Accuracy tells us how many predictions are correct out of all predictions.

$$\text{Accuracy} = (TP+TN) / (TP+TN+FP+FN)$$

Precision

Precision measures how many of the instances predicted as positive are actually positive.

In other words - Out of everything the model predicted as Positive, how many were actually Positive?

Precision = TP / (TP+FP)

Recall

Recall measures how many of the actual positive instances were correctly identified by the model.

In other words - Out of all the actual Positive cases, how many did the model correctly identify?

$$\text{Recall} = TP / (TP+FN)$$

F1-Score

F1-Score is the harmonic mean of Precision and Recall.

It combines Precision and Recall into a single metric.

F1-Score = 2 * (Precision * Recall) / (Precision + Precision)

Binary Predictive Classification

Binary classification is a type of supervised machine learning problem in which the model predicts one of **two possible classes**.

Example:

E-Mail classification – Spam or Not

Sensitivity

Sensitivity measures the ability of a classification model to correctly identify positive cases.

In other words - Sensitivity tells us how many of the actual positive cases were correctly detected by the model.

$$Sensitivity = \frac{\text{Correctly detected Positive Cases}}{\text{Total Actual Positive Cases}}$$

Sensitivity = Recall

Sensitivity → **Positive**

Specificity

Specificity measures the ability of a classification model to correctly identify negative cases.

In other words - Specificity tells us how many of the actual negative cases were correctly identified by the model.

$$Specificity = \frac{\text{Correctly detected Negative Cases}}{\text{Total Actual Negative Cases}}$$

Specificity → **Negative**

References:

Textbook

1. Ethem Alpaydin, Introduction to Machine Learning, Fourth Edition, MIT Press, 2020
2. R for Data Science by Hadley Wickham & Garrett Grolemund (Comprehensive R programming guide)
3. Python for Data Analysis by Wes McKinney (Covers Pandas, NumPy, and Jupyter, written by the creator of Pandas)
4. Pattern Recognition and Machine Learning by Christopher M. Bishop (Covers deep math and algorithms)

Reference book

1. Hadley Wickham, Garrett Grolemund, R for data science : Import, Tidy, Transform, Visualize, And Model Data Paperback, 2017.
2. Han, J., Kamber, M., Pei, J. Data mining concepts and techniques. Morgan Kaufmann. 2011
3. James, G., Witten, D., T., Tibshirani, R. An Introduction to statistical learning with applications in R. Springer. 2013

Parul® University | NAAC GRADE A++

https://paruluniversity.ac.in/

